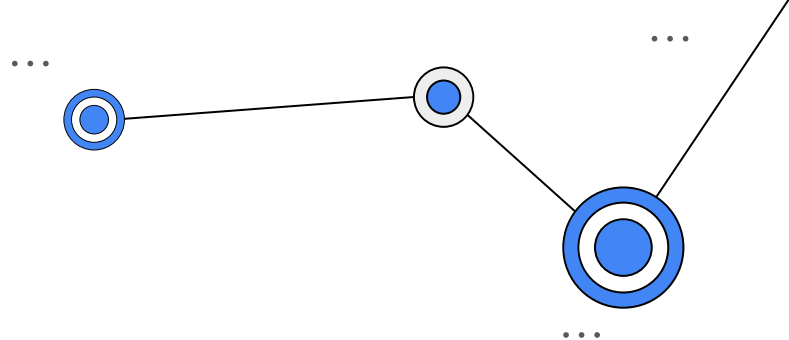




UNIVERSITÀ
DEGLI STUDI
DI TRIESTE



440MI

DATA-DRIVEN SYSTEMS ENGINEERING

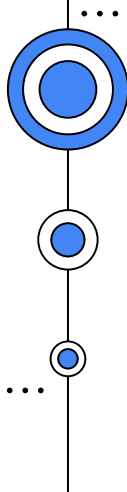
L10 - Software Processes

Sylvio Barbon Junior
sylvio.barbonjunior@units.it

10

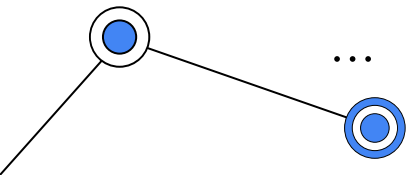
Software Engineering

Software Processes

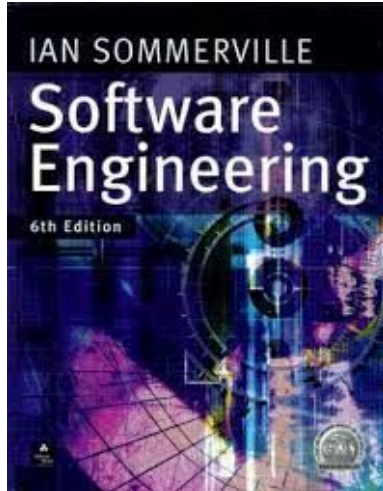
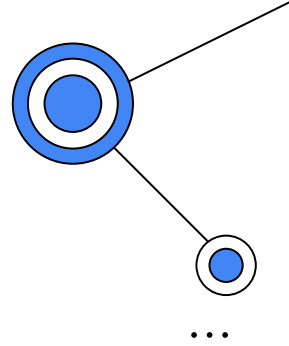


UNIVERSITÀ
DEGLI STUDI
DI TRIESTE





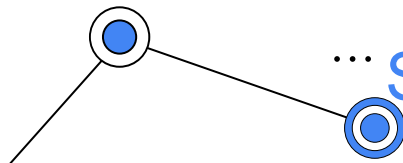
Last lesson



... Introduction to Software Engineering



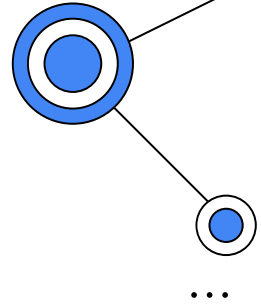
**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**



... SE 10 - Software Processes

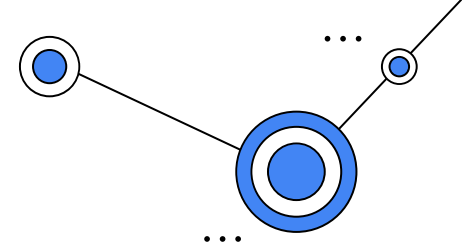
Today's goal:

- Software process models
- Process activities
- Coping with change
- The Rational Unified Process



SE 10 - Software Processes

Software process descriptions

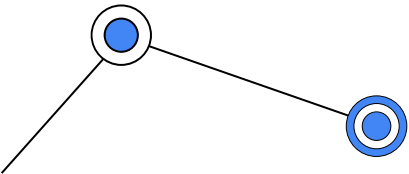


✧ When we describe and discuss software project **processes**, we usually talk about the **activities** in these processes such as:

- * specifying a data model
- * designing a user interface
- * etc.

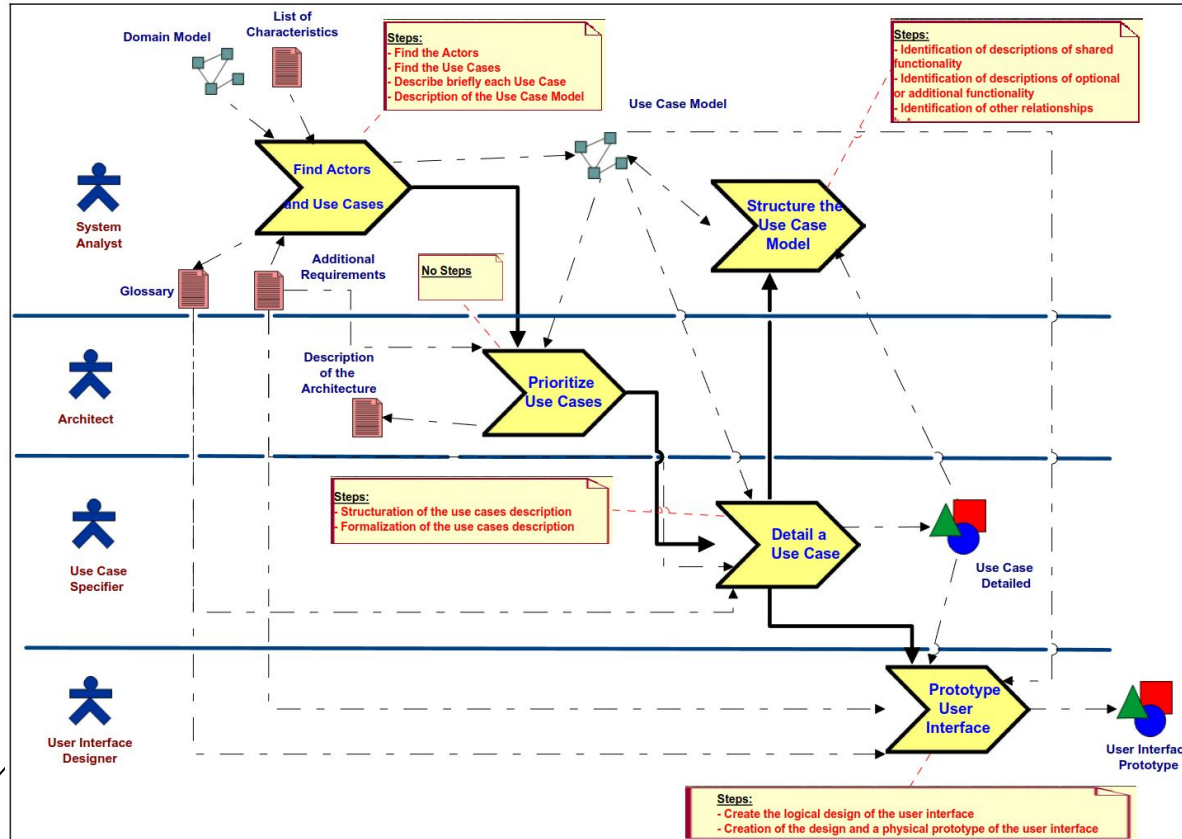
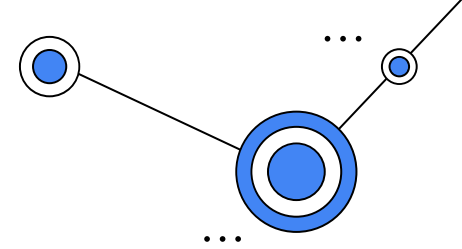
✧ **Process descriptions** may also include:

- * **Products**, which are the outcomes of a process activity;
- * **Roles**, which reflect the responsibilities of the people involved in the process;
- * **Pre- and post-conditions**, which are statements that are true before and after a process activity has been enacted or a product produced.



SE 10 - Software Processes

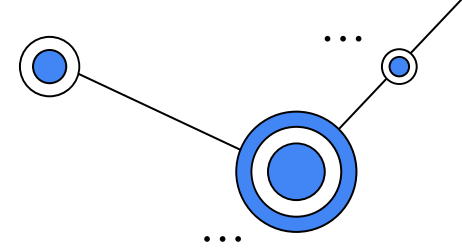
Software process descriptions



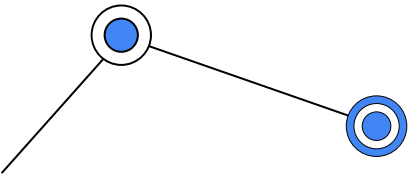
https://www.researchgate.net/publication/300160673_Metrics_for_software_process_models/

SE 10 - Software Processes

Plan-driven and agile processes

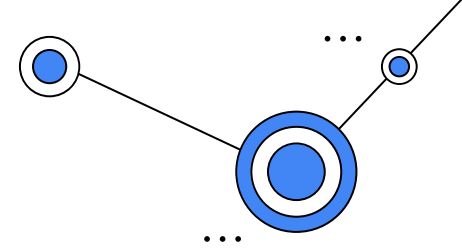


- ✧ **Plan-driven** processes are processes where all of the process activities are planned in advance and progress is measured against this plan.
- ✧ In **agile** processes, planning is incremental and it is easier to change the process to reflect changing customer requirements.
- ✧ In practice, most practical processes include elements of both plan-driven and agile approaches.
- ✧ There are no right or wrong software processes.

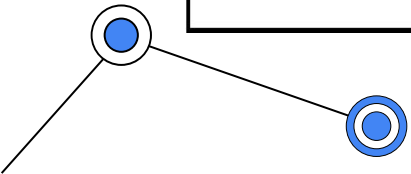


SE 10 - Software Processes

Plan-driven and agile processes

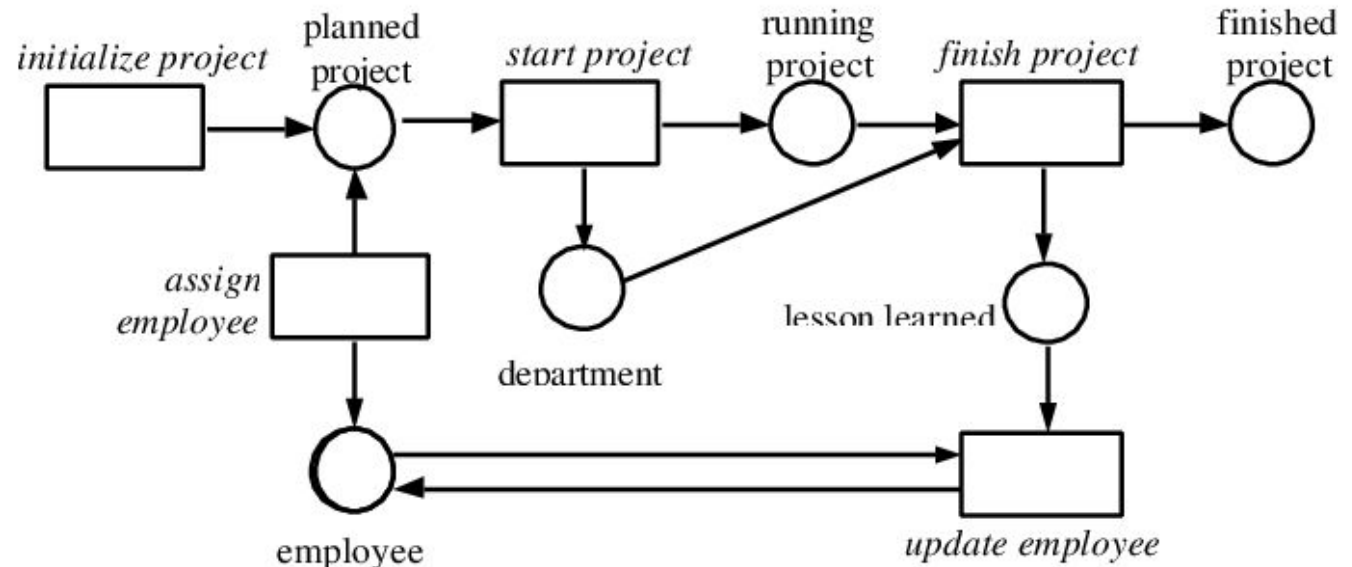


Agile Process	Plan Driven Processes
It's suitable for small products and teams, as its limited scalability	It's suitable for large products and teams, as it's hard to scale down
It's untested on safety-critical products	It can handle highly critical products
It's good for dynamic, but expensive for stable environments.	It's good for stable, but expensive for dynamic environments
It requires experienced Agile personnel throughout	It requires experienced personnel only at start if environment is stable
Personnel thrive on freedom and chaos	Personnel thrive on structure and order



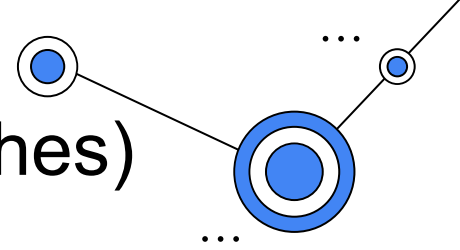
SE 10 - Software Processes

Software Process Model is a simplified representation of a software process!



SE 10 - Software Processes

Software process models (main approaches)



✧ The waterfall model

Plan-driven model. Separate and distinct phases of specification and development.

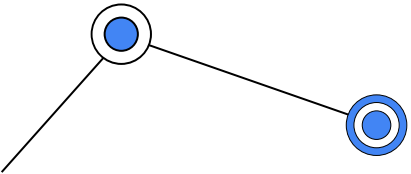
✧ Incremental development model

Specification, development and validation are interleaved. May be **plan-driven** or **agile**.

✧ Reuse-oriented software engineering model

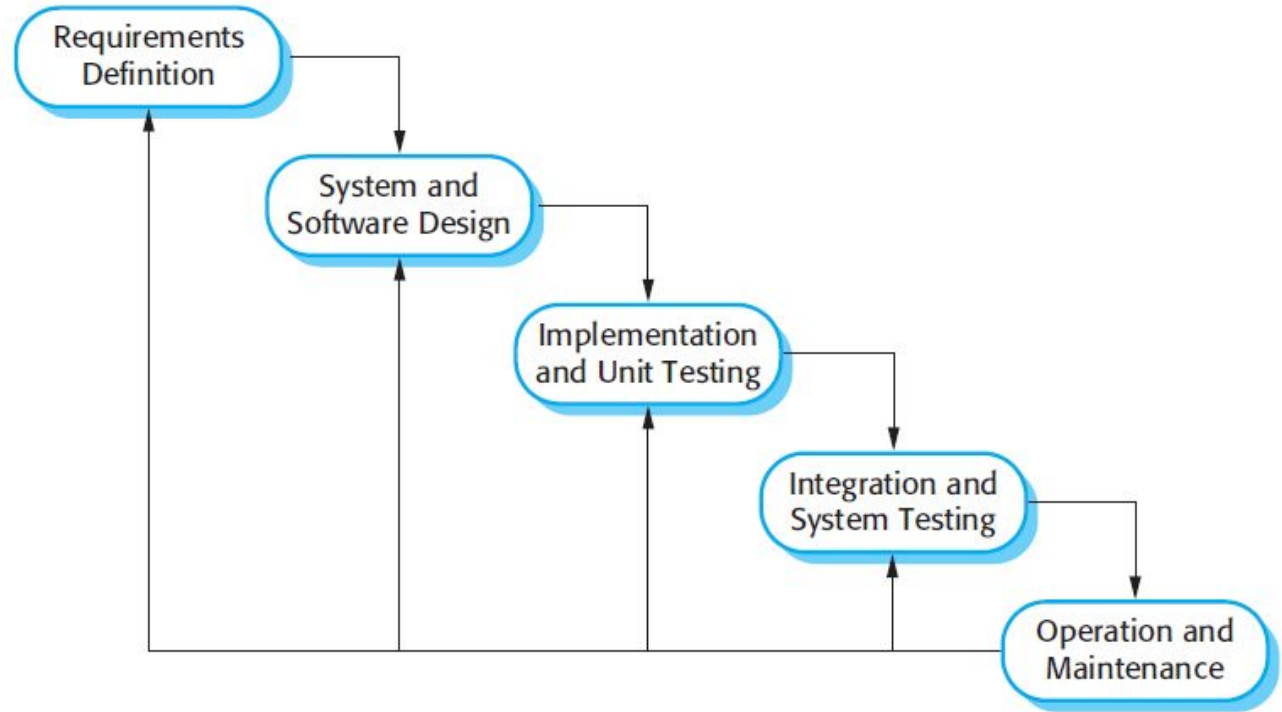
The system is assembled from existing components. May be **plan-driven** or **agile**.

✧ In practice, most large systems are developed using a process that incorporates elements from all of these models.



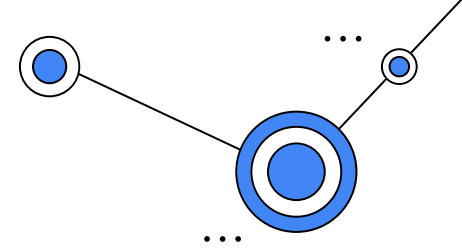
SE 10 - Software Processes

The waterfall model (1970)



SE 10 - Software Processes

Waterfall model phases



✧ There are separate identified phases in the waterfall model:

Requirements analysis and definition (users interaction)

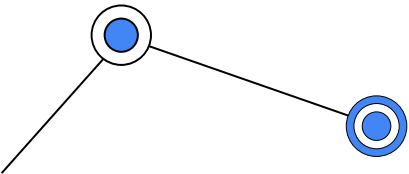
System and software design (hardware, software and communication)

Implementation and unit testing (programs, libraries and how to test them)

Integration and system testing (complete SW System test, meeting the req.)

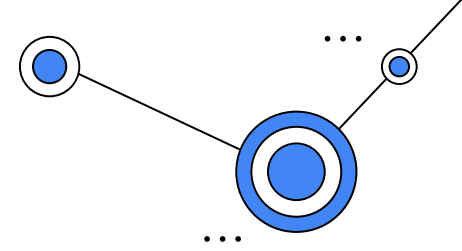
Operation and maintenance (correcting errors, updating, addressing new req.)

✧ The main **drawback** of the waterfall model **is the difficulty of accommodating change after** the process is underway. In principle, a phase has to be complete before moving onto the next phase.

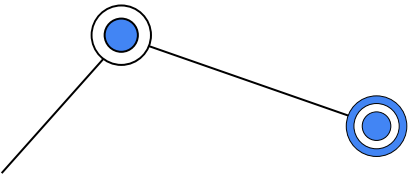


SE 10 - Software Processes

Waterfall model **problems**

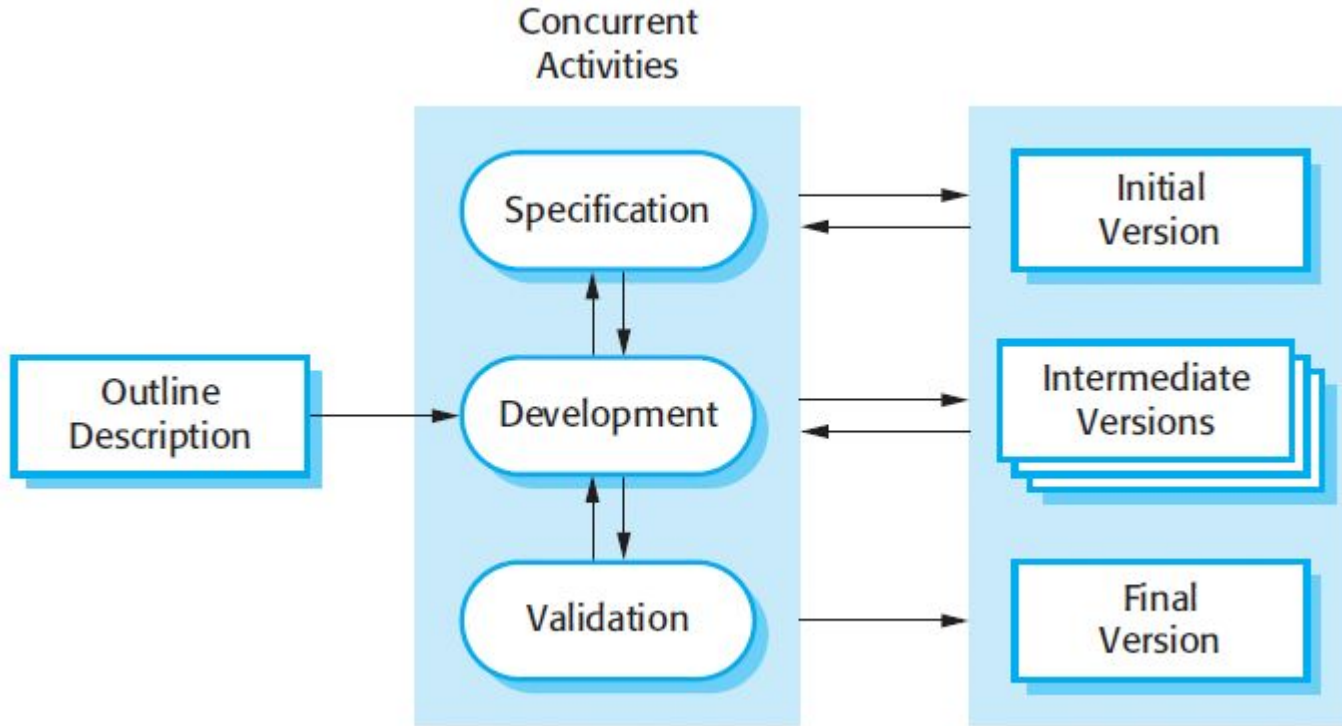


- ✧ **Inflexible partitioning of the project** into distinct stages makes it difficult to respond to changing customer requirements.
 - * Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
 - * Few business systems have stable requirements.
- ✧ The waterfall model is mostly used for **large *static* systems engineering** projects where a system is developed at several sites.
 - In those circumstances, the plan-driven nature of the waterfall model **helps coordinate the work.**



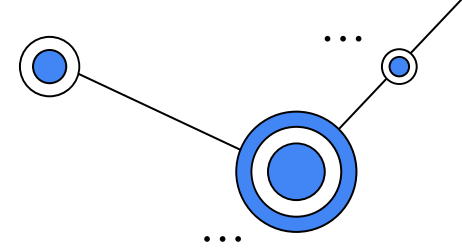
SE 10 - Software Processes

Incremental development

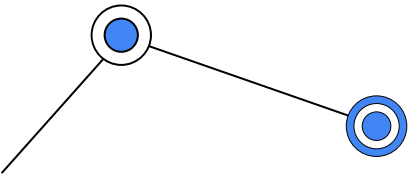


SE 10 - Software Processes

Incremental development benefits

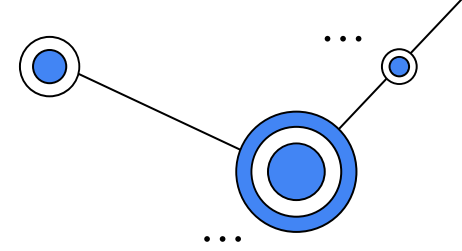


- ✧ The **cost** of accommodating **changing** customer requirements **is reduced**.
The amount of analysis and documentation that has to be **redone is much less** than is required with the waterfall model.
- ✧ **It is easier to get** customer **feedback** on the development work that has been done.
Customers can comment on demonstrations of the software and see how much has been implemented.
- ✧ **More rapid delivery and deployment of useful software** to the customer is possible.
Customers are able to use and gain value from the software earlier than is possible with a waterfall process.



SE 10 - Software Processes

Incremental development problems

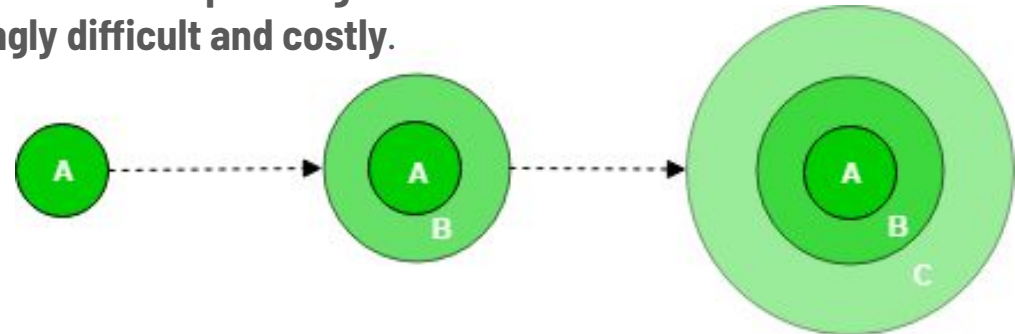
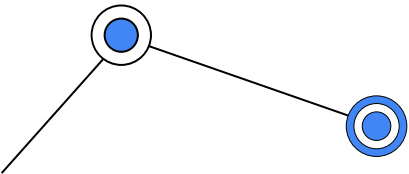


✧ The **process is not visible**.

Managers need regular deliverables to measure progress. If systems are developed quickly, **it is not cost-effective to produce documents** that reflect every version of the system.

✧ **System structure tends to degrade** as **new increments** are added.

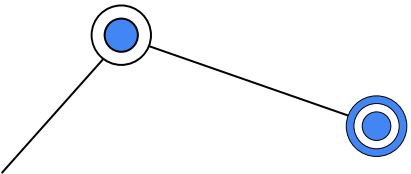
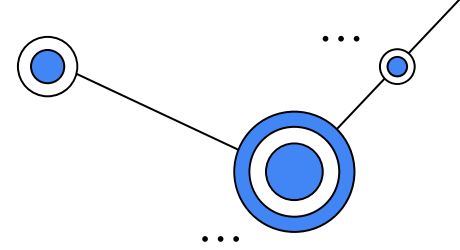
Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. **Incorporating further software changes becomes increasingly difficult and costly.**



SE 10 - Software Processes

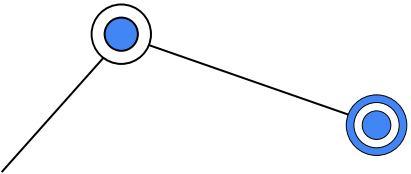
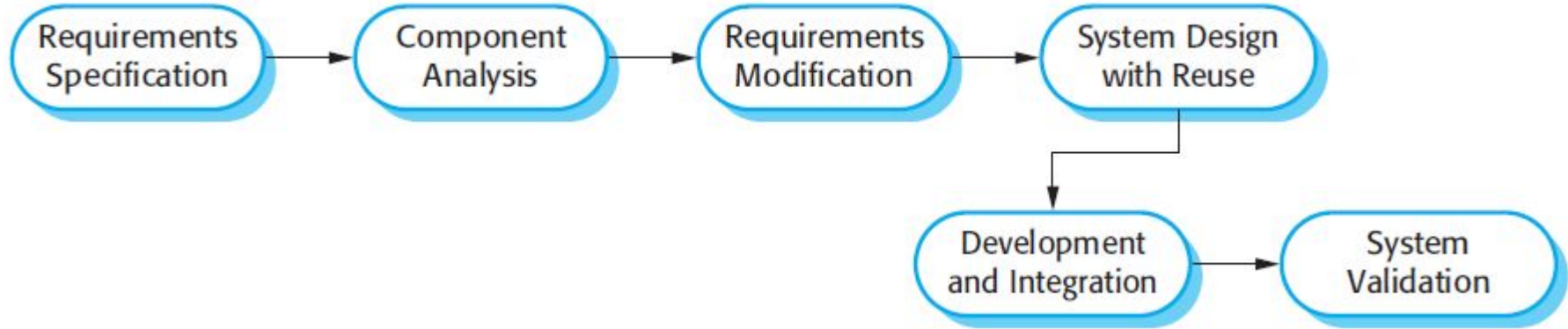
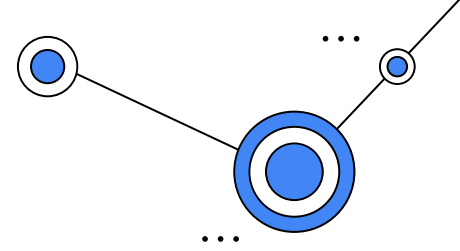
Reuse-oriented software engineering

- ✧ Based on systematic reuse where systems are integrated from existing components or **COTS (Commercial-off-the-shelf)** systems.
- ✧ Process stages
 - Component analysis;**
 - Requirements modification;**
 - System design with reuse;**
 - Development and integration.**
- ✧ Reuse is now the standard approach for building many types of business system



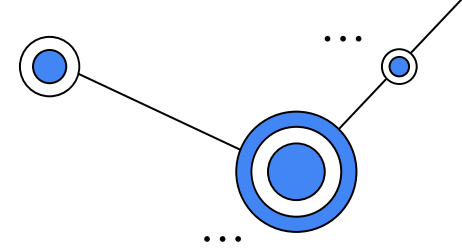
SE 10 - Software Processes

Reuse-oriented software engineering



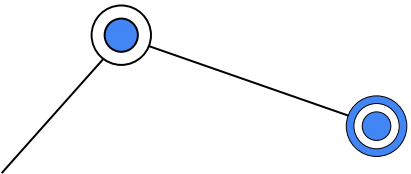
SE 10 - Software Processes

Reuse-oriented software engineering

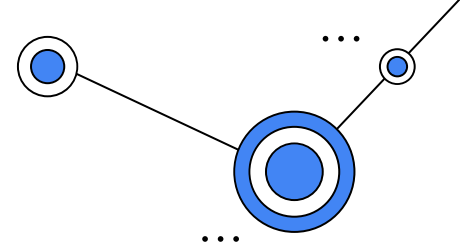


✧ Types of software component:

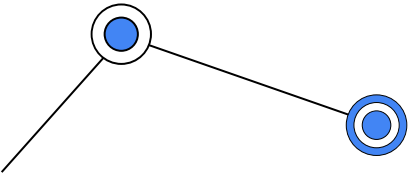
- * Web services that are developed according to service standards and which are available for remote invocation.
- * Collections of objects that are developed as a package to be integrated with a component framework such as .NET or J2EE.
- * Stand-alone software systems (COTS) that are configured for use in a particular environment.



SE 10 - Software Processes

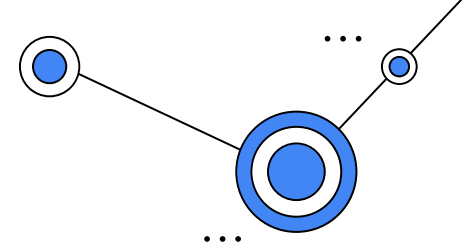


Process activities



SE 10 - Software Processes

Process activities

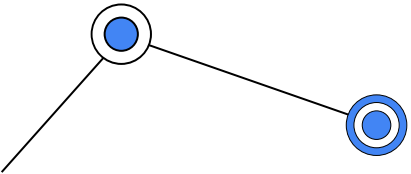


✧ Real software **processes are** interleaved **sequences** of:

- * technical;
- * collaborative;
- * managerial activities;

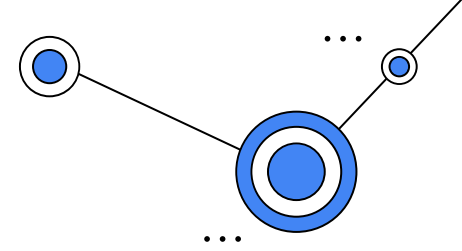
✧ with the overall **goal** of:

- * specifying;
- * designing;
- * implementing;
- * testing a software system.



SE 10 - Software Processes

Process activities



✧ The four basic **process activities** of:

* specification;

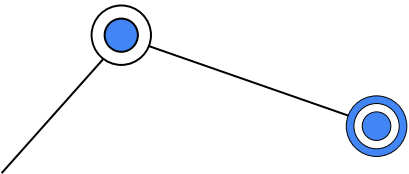
* development (design and implementation);

* validation;

* evolution;

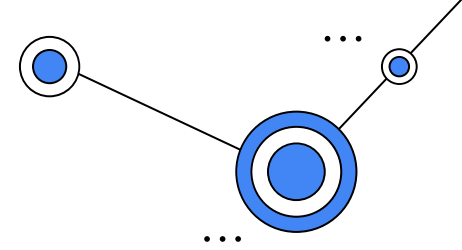
✧ are organized differently in different **development processes**.

✧ In the waterfall model, they are organized in sequence, whereas in incremental development they are interleaved.

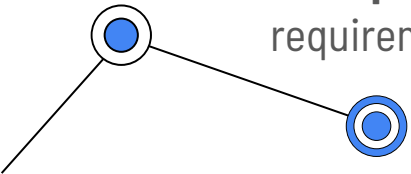


SE 10 - Software Processes

Software **specification**



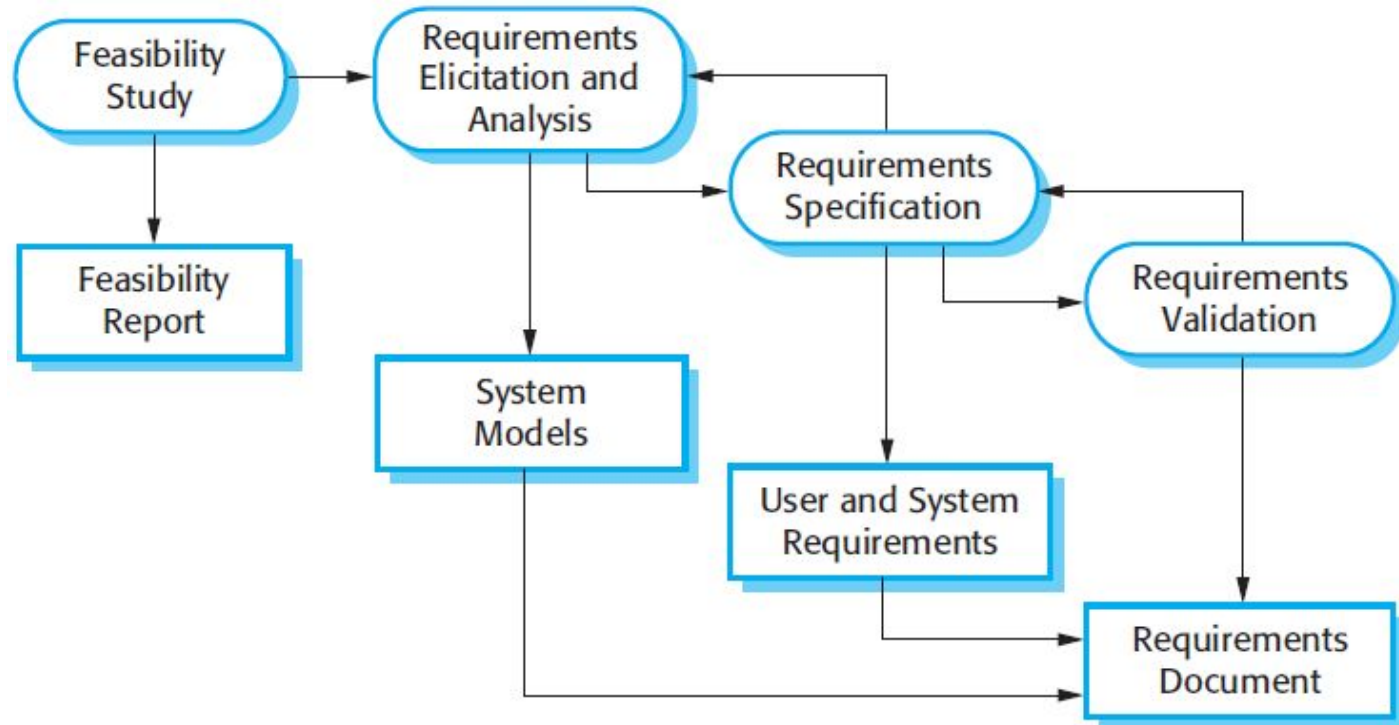
- ✧ The process of establishing what services are required and the constraints on the system's operation and development.
- ✧ Requirements engineering process:
 - 1) **Feasibility** study: Is it technically and financially feasible to build the system?
 - 2) **Requirements elicitation and analysis**: What do the system stakeholders require or expect from the system?
 - 3) **Requirements specification**: Defining the requirements in detail.
 - 4) **Requirements validation**: Checking the validity of the requirements.



SE 10 - Software Processes

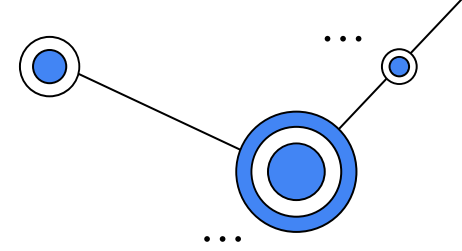
Software **specification**

The requirements engineering process:

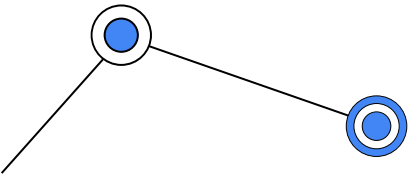


SE 10 - Software Processes

Software design and implementation



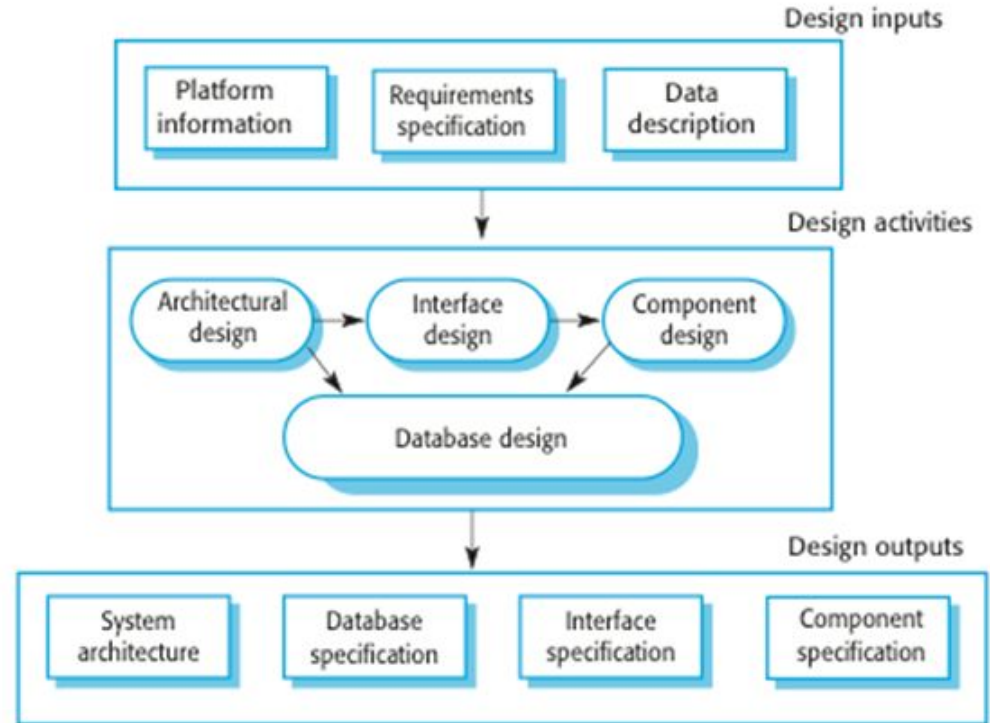
- ✧ The process of converting the system specification into an executable system.
- ✧ Software design
 - Design a software structure that realises the specification;
- ✧ Implementation
 - Translate this structure into an executable program;
- ✧ The activities of design and implementation are closely related and may be interleaved.



SE 10 - Software Processes

Software design and implementation

A general model of the design process

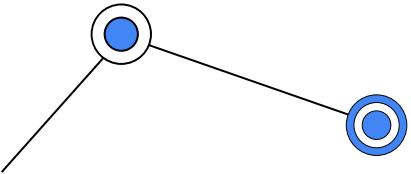
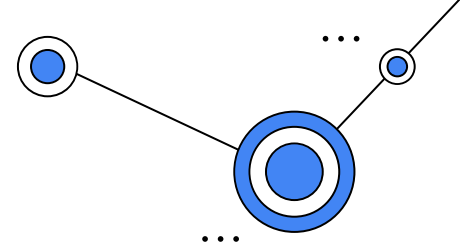


SE 10 - Software Processes

Software design and implementation

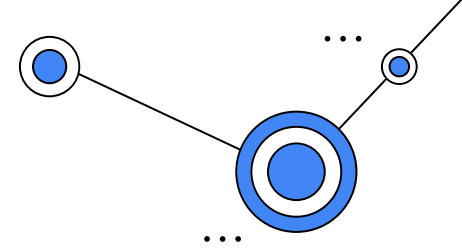
Design activities

- ✧ **Architectural design**, where you identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships and how they are distributed.
- ✧ **Interface design**, where you define the interfaces between system components.
- ✧ **Component design**, where you take each system component and design how it will operate.
- ✧ **Database design**, where you design the system data structures and how these are to be represented in a database.

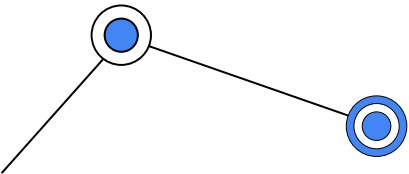


SE 10 - Software Processes

Software validation



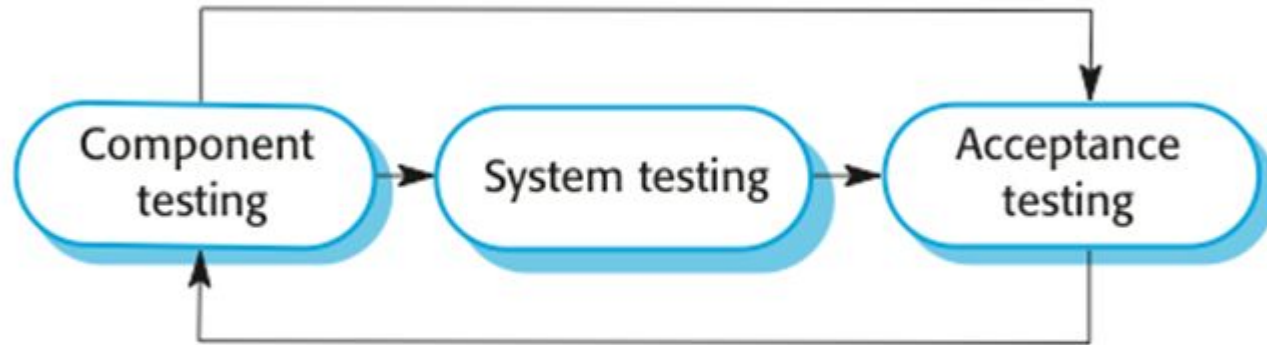
- ✧ **Verification and validation** (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer.
- ✧ Involves checking and review processes and system testing.
- ✧ System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.
- ✧ **Testing** is the most commonly used V & V activity.



SE 10 - Software Processes

Software validation

Stages of testing



SE 10 - Software Processes

Software validation

Stages of testing

✧ Development or component testing

Individual components are tested independently;

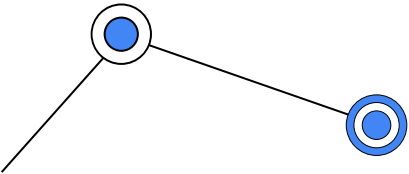
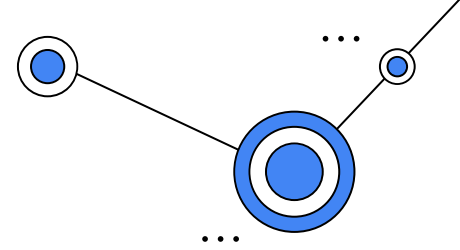
Components may be functions or objects or coherent groupings of these entities.

✧ System testing

Testing of the system as a whole. Testing of emergent properties is particularly important.

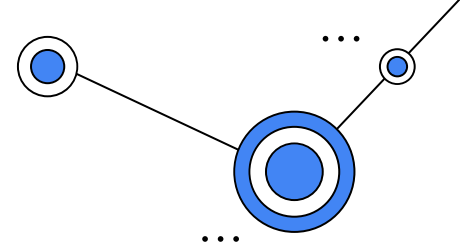
✧ Acceptance testing

Testing with customer data to check that the system meets the customer's needs.

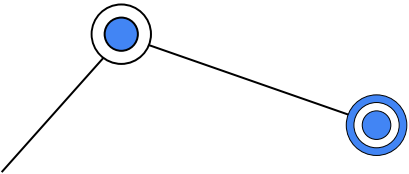


SE 10 - Software Processes

Software **evolution**

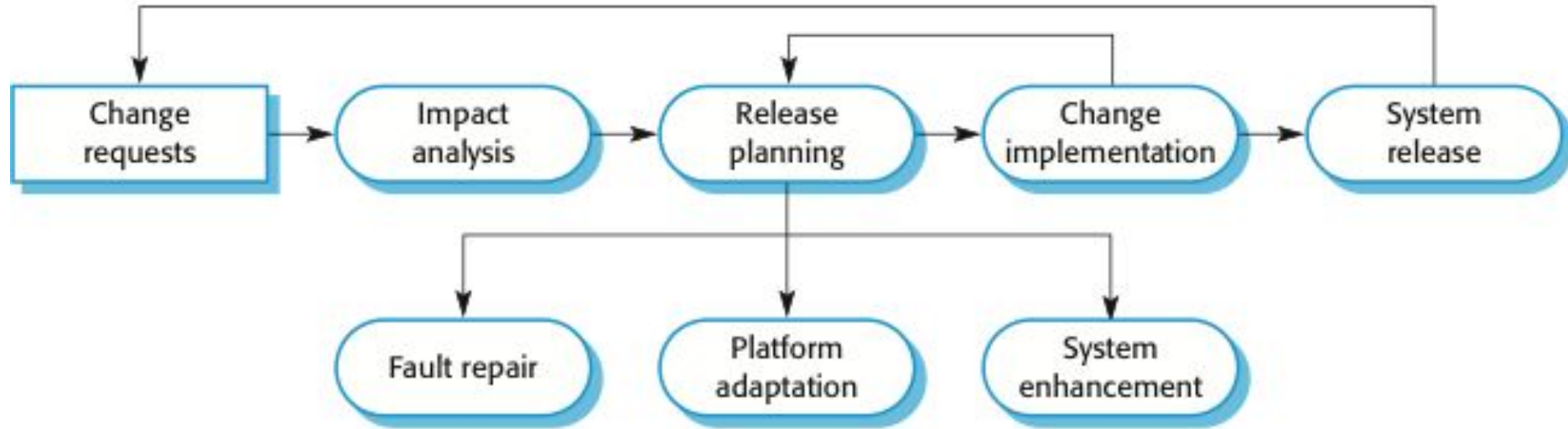


- ✧ Software is inherently **flexible** and can **change**.
- ✧ As requirements change through changing business circumstances, the software that supports the business must also evolve and change.
- ✧ Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.



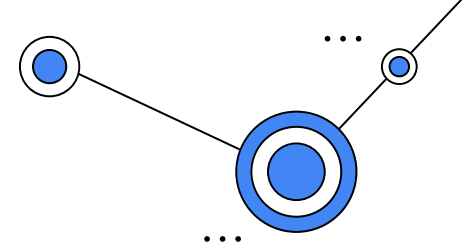
SE 10 - Software Processes

System evolution

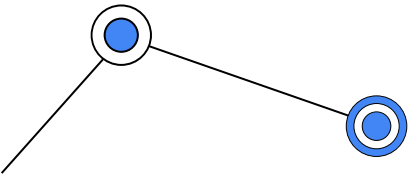


SE 10 - Software Processes

Key points

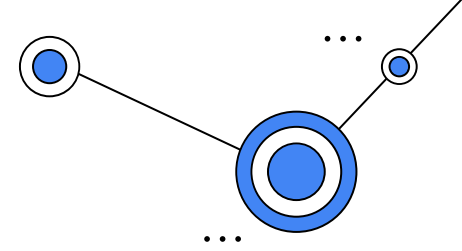


- ✧ **Software processes** are the activities involved in producing a software system.
- ✧ **Software process models** are **abstract** representations of these processes.
- ✧ **General process models** describe the organization of software processes. Examples of these general models include the 'waterfall' model, incremental development, and reuse-oriented development.

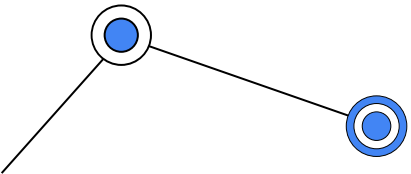


SE 10 - Software Processes

Key points

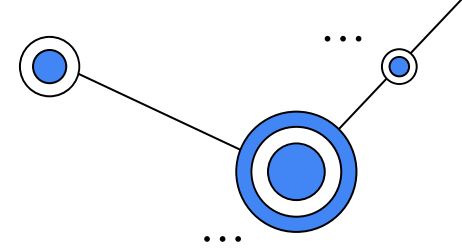


- ✧ **Requirements engineering** is the process of developing a software specification.
- ✧ **Design and implementation** processes are concerned with transforming a requirements specification into an executable software system.
- ✧ **Software validation** is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.
- ✧ **Software evolution** takes place when you change existing software systems to meet new requirements. The software must evolve to remain useful.

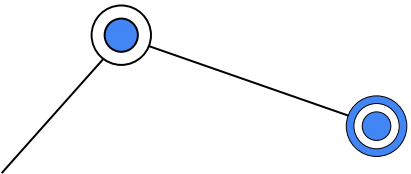


SE 10 - Software Processes

Coping with change

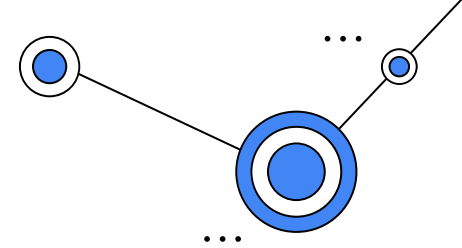


- ✧ Change is inevitable in all large software projects.
 - * Business changes lead to new and changed system requirements;
 - * New technologies open up new possibilities for improving implementations;
 - * Changing platforms require application changes;
- ✧ Change leads to **rework** so the **costs** of change include both rework (**e.g. re-analysing requirements**) as well as the costs of implementing **new functionality**



SE 10 - Software Processes

Reducing the costs of rework



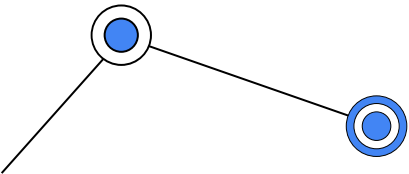
- 1) Change **avoidance**, where the software process includes activities that can anticipate possible changes before significant rework is required.

For example, a **prototype** system may be developed to show some key features of the system to customers.

- 2) Change **tolerance**, where the process is designed so that changes can be accommodated at relatively low cost.

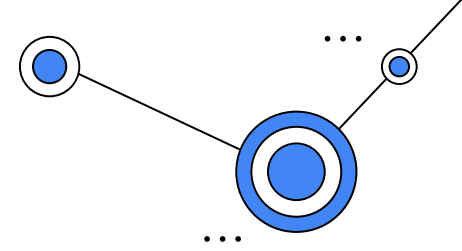
This normally involves some form of **incremental development**.

Proposed changes may be implemented in increments that have not yet been developed. If this is impossible, then only **a single increment** (a small part of the system) may have be altered to **incorporate the change**.

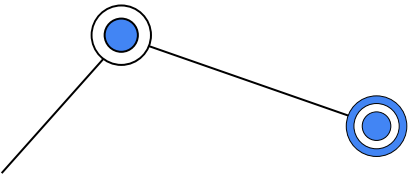


SE 10 - Software Processes

Software **prototyping**



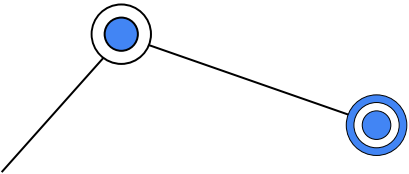
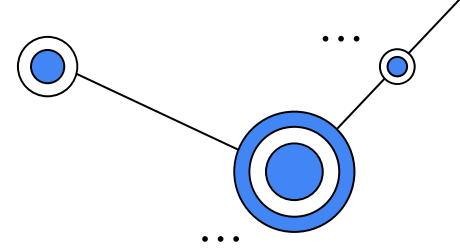
- ✧ A **prototype** is an initial version of a system used to demonstrate concepts and try out design options.
- ✧ A prototype **can be used** in:
 - 1) The requirements engineering process to help with **requirements elicitation and validation**;
 - 2) In design processes to explore options and develop a **UI design**;
 - 3) In the testing process to run **back-to-back tests** (two or more components of different versions are tested with the same series of data at different conditions).



SE 10 - Software Processes

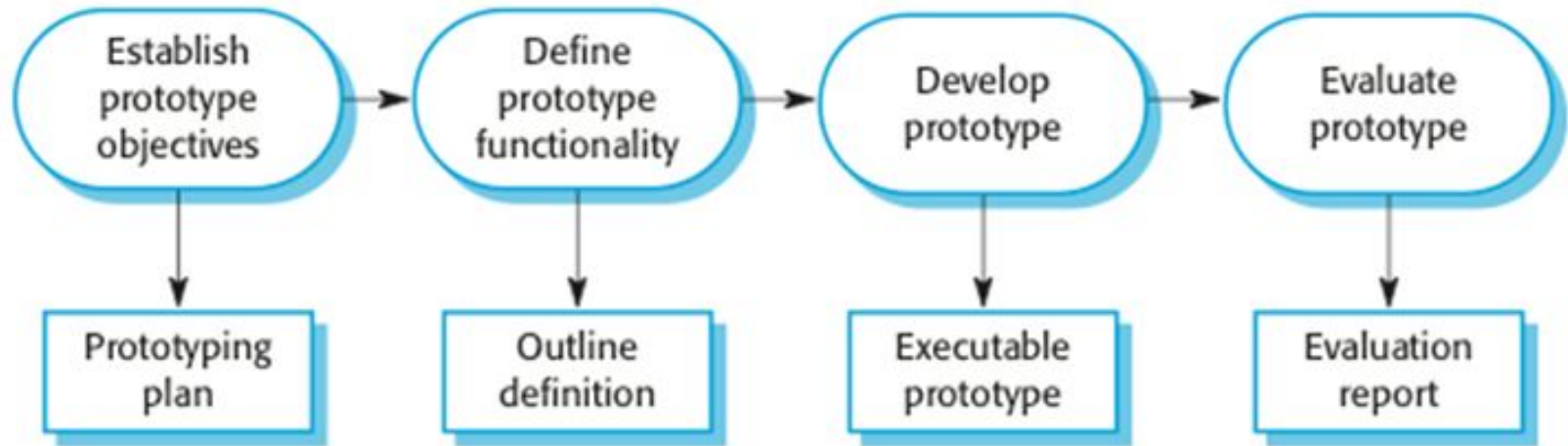
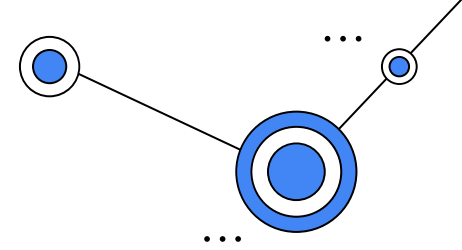
Benefits of **prototyping**

- ✧ Improved system **usability**.
- ✧ A closer **match** to users' **real needs**.
- ✧ **Improved design quality**.
- ✧ **Improved maintainability**.
- ✧ **Reduced development** effort.



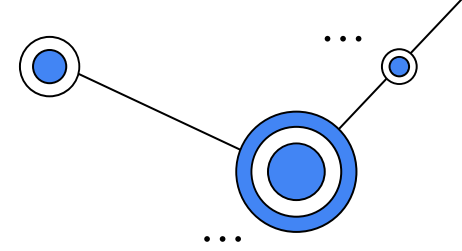
SE 10 - Software Processes

The process of **prototype** development

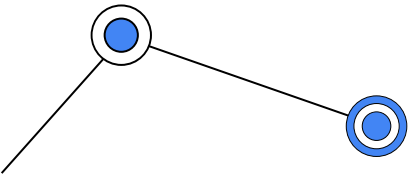


SE 10 - Software Processes

Prototype development

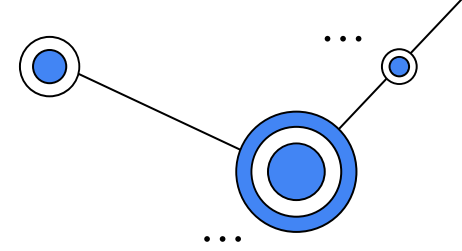


- ✧ May be based on **rapid prototyping languages or tools**;
- ✧ May involve **leaving** out functionality
 - 1) Prototype should **focus on** areas of the product that are not **well-understood**;
 - 2) **Error checking** and **recovery** may **not** be included in the prototype;
 - 3) **Focus on functional** rather than non-functional requirements such as reliability and security



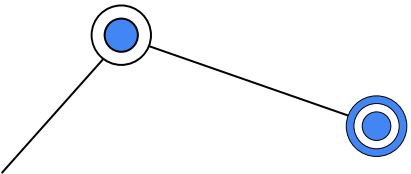
SE 10 - Software Processes

Throw-away prototypes



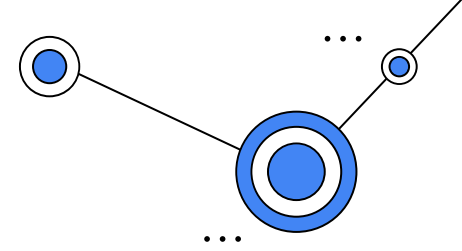
✧ Prototypes **should be discarded after development** as they are not a good basis for a production system:

- 1) It may be **impossible** to tune the system **to meet non-functional** requirements;
- 2) Prototypes **are normally undocumented**;
- 3) The prototype structure **is usually degraded** through rapid **change**;
- 4) The prototype probably **will not meet** normal organisational **quality standards**.

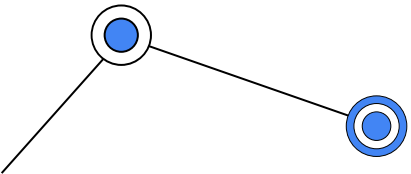


SE 10 - Software Processes

Incremental delivery

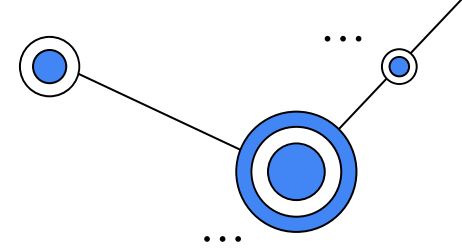


- ✧ Rather than deliver the system as a single delivery, the development and delivery is **broken down into increments** with each increment delivering part of the required functionality.
- ✧ **User requirements are prioritised** and the highest priority requirements are included in **early increments**.
- ✧ Once the development of an increment is started, the **requirements are frozen** though requirements for **later increments can continue to evolve**.



SE 10 - Software Processes

Incremental development and delivery

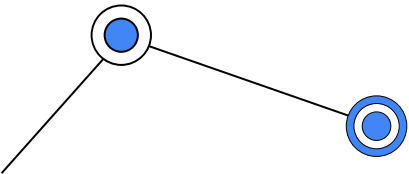


✧ Incremental development

- 1) **Develop the system in increments and evaluate** each increment before proceeding to the development of the next increment;
- 2) Normal approach used in **agile methods**;
- 3) **Evaluation done by user**/customer proxy (representative).

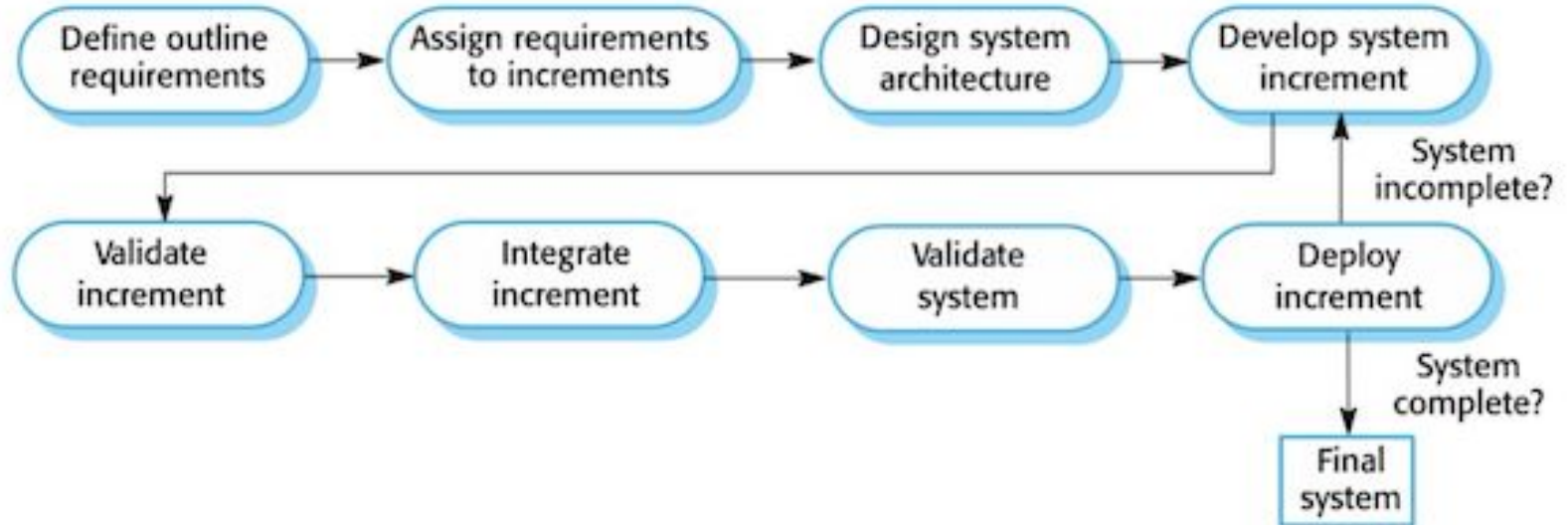
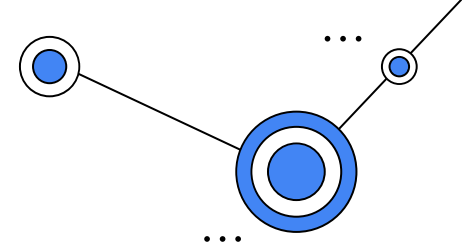
✧ Incremental delivery

- 1) **Deploy** an increment for use by **end-users**;
- 2) More **realistic evaluation** about practical use of software;
- 3) **Difficult** to implement for replacement systems as **increments have less functionality** than the system being **replaced**.



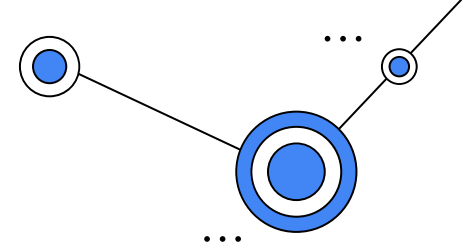
SE 10 - Software Processes

Incremental delivery

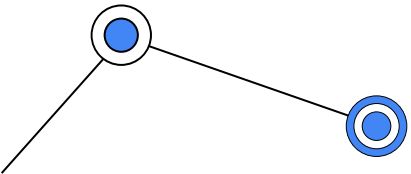


SE 10 - Software Processes

Incremental delivery advantages

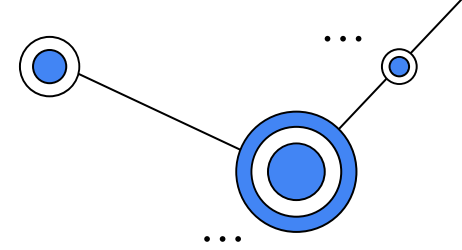


- ✧ **Customer** value can be delivered with each increment so system functionality **is available earlier**.
- ✧ **Early increments** act as a prototype to **help elicit requirements** for later increments.
- ✧ **Lower risk** of overall project **failure**.
- ✧ **The highest priority** system services tend to receive **the most testing**.



SE 10 - Software Processes

Incremental delivery problems

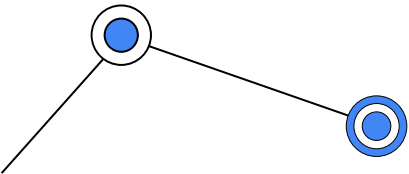


- ✧ Most systems require a set of basic facilities that are used by different parts of the system.

As requirements are not defined in detail until an increment is to be implemented, it can be **hard to identify common facilities that are needed by all increments.**

- ✧ The **essence of iterative processes** is that the specification is developed in **conjunction** with the software.

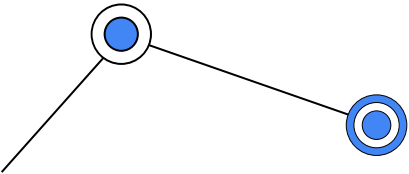
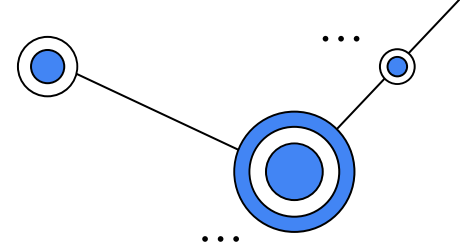
However, this conflicts with the procurement model of many organizations, where the complete system specification **is part of the system development contract.**



SE 10 - Software Processes

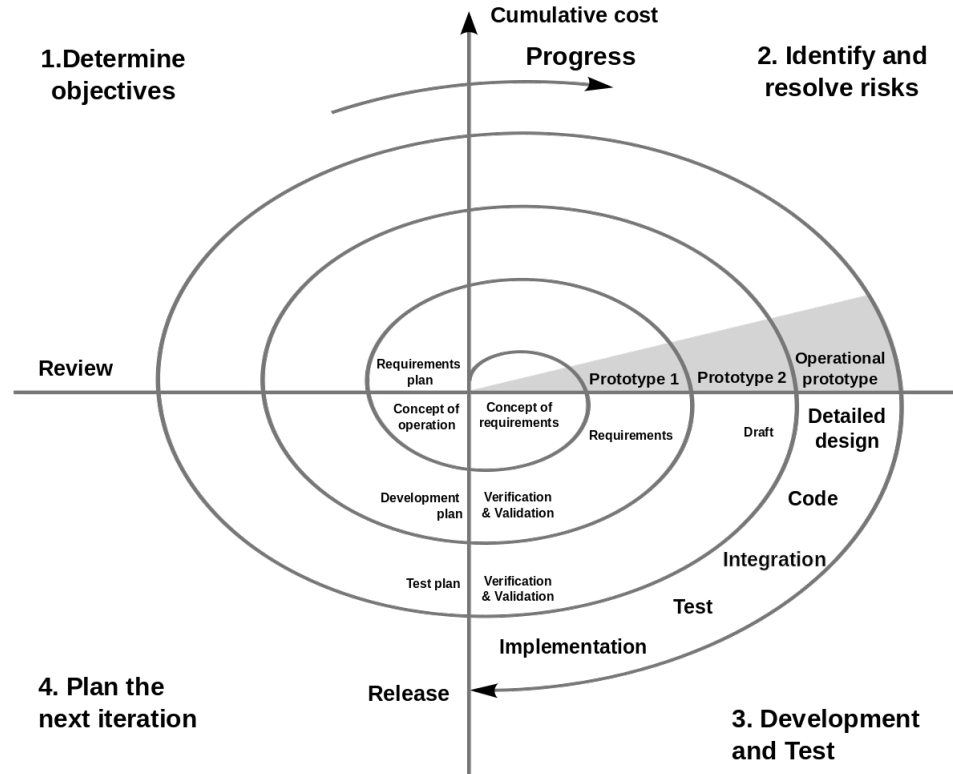
Boehm's spiral model

- ✧ Process is represented as **a spiral** rather than as a sequence of activities with backtracking.
- ✧ **Each loop** in the spiral represents **a phase** in the process.
- ✧ **No fixed phases** such as specification or design - loops in the spiral are **chosen depending** on what is required.
- ✧ **Risks are explicitly assessed** and resolved throughout the process.



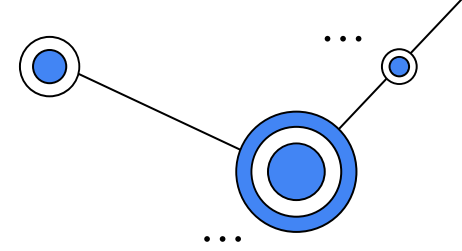
SE 10 - Software Processes

Boehm's spiral model of the software process

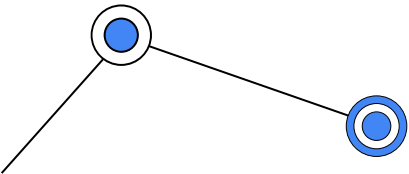


SE 10 - Software Processes

Spiral model sectors



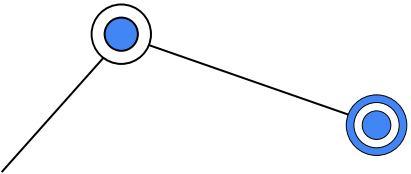
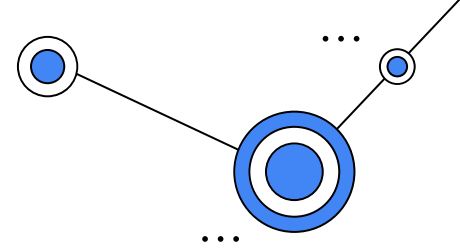
- ✧ **Objective** setting: Specific objectives for the phase are identified.
- ✧ **Risk** assessment and reduction: Risks are assessed and activities put in place to reduce the key risks.
- ✧ **Development** and **validation**: A development model for the system is chosen which can be any of the generic models.
- ✧ **Planning**: The project is reviewed and the next phase of the spiral is planned.



SE 10 - Software Processes

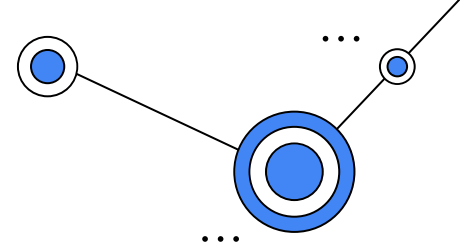
Spiral model usage

- ✧ Spiral model has been very influential in helping people think about iteration in software processes and introducing the risk-driven approach to development.
- ✧ In practice, however, the model is rarely used as published for practical software development.



SE 10 - Software Processes

The Rational Unified Process

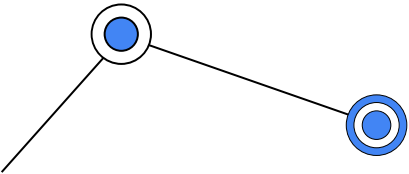


- ✧ A **modern** generic process derived from the work on the **UML** and associated process.
- ✧ Brings together aspects of the 3 generic process models discussed previously.
- ✧ Normally described from 3 perspectives

A dynamic perspective that shows phases **over time**;

A static perspective that shows process **activities**;

A practice perspective that suggests **good practice**.



SE 10 - Software Processes

RUP good practice

✧ Develop software iteratively

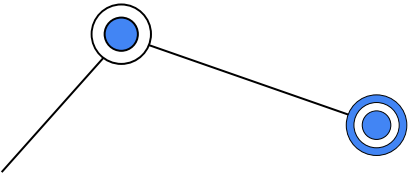
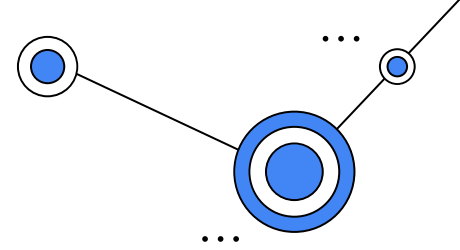
Plan increments based on customer priorities and deliver highest priority increments first.

✧ Manage requirements

Explicitly document customer requirements and keep **track of changes** to these requirements.

✧ Use component-based architectures

Organize the system architecture as a set of **reusable components**.



SE 10 - Software Processes

RUP good practice

✧ Visually model software

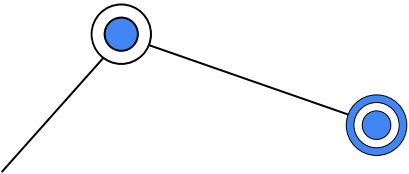
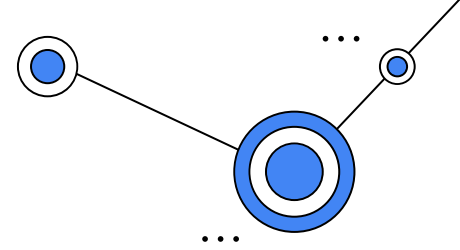
Use **graphical UML models** to present static and dynamic views of the software.

✧ Verify software quality

Ensure that the software meet's organizational quality standards.

✧ Control changes to software

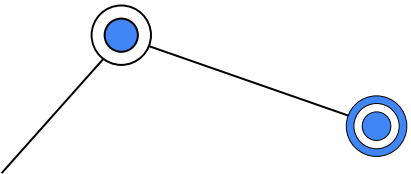
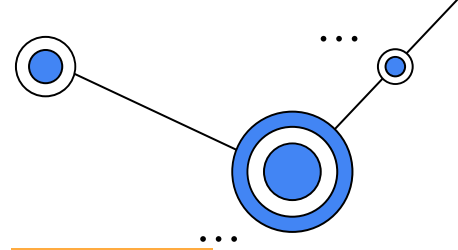
Manage software changes **using a change management system** and **configuration management tools**.



SE 10 - Software Processes

Key points

- ✧ **Processes should include activities to cope with change.** This may involve a **prototyping** phase that helps **avoid poor decisions** on requirements and design.
- ✧ **Processes** may be structured for **iterative development** and delivery so that changes may be made without disrupting the system as a whole.
- ✧ The **Rational Unified Process** is a modern generic process model that is organized into phases but separates activities (requirements, analysis and design, etc.) from these phases.



SE 10 - Software Processes

The end!

